Object in C#

- Object class or System.Object is the ultimate base class in C#. All value types(System.ValueType) and reference types derive from System.Object.
- Common practice for implementing generic routines has been to write a method that accepts arguments of type object and allow users to pass in any type they want.
- This approach is difficult to maintain because of the lack of type safety and inefficiency for value types.

Virtual methods of Object class

- Has various virtual methods
  - Equals(Object)
  - Equals(Object, Object)
  - ReferenceEquals(Object, Object)
  - GetType()
  - GetHashCode()
  - ToString()
  - MemberwiseClone()

Equals

- The Equals method of the object class provides a default implementation that compares two reference type objects for reference equality and two value type objects for content equality.
- Reference equality occurs when two reference type objects refer to the same object.
- Equals looks for reference equality. It can be overloaded to make it value equality for reference types.

```
public class Person{
    public string name {get; set;}
    public int    age  {get; set;}
    public Person(string name, int age){
       this.name = name;
       this.age  = age;
    }

}

public class GFG{
    static public void Main (){
        Person p1 = new Person("Avishek", 10);
        Person p2 = new Person("Avishek", 10);

        //p1 and p2 points to different memory locations
        Console.WriteLine("Reference type Equals: {0}", p1.Equals(p2));

        //Now p3 points to same memory location as p1.
        Person p3 = p1;
        Console.WriteLine("Reference type Equals: {0}", p1.Equals(p3));

        int i = 10;
        int j = 10;
        Console.WriteLine("Value type Equals: {0}",i.Equals(j));
    }
}
```

```
Reference type Equals: False
Reference type Equals: True
Value type Equals: True
```

Override Equals

```
public class Person{
    public string name {get; set;}
    public int    age  {get; set;}
    public Person(string name, int age){
       this.name = name;
       this.age  = age;
    }
    //override the Equals
    public override bool Equals(object o){
        if(o == null)
            return false;

        if(this.GetType() != o.GetType())
            return false;

        Person p = (Person)o;
        return this.name == p.name && this.age == p.age;
    }
}

public class GFG{
    static public void Main (){
        Person p1 = new Person("Avishek", 10);
        Person p2 = new Person("Avishek", 10);

        //p1 and p2 points to different memory locations
        Console.WriteLine("Reference type Equals: {0}", p1.Equals(p2));

        //Now p3 points to same memory location as p1.
        Person p3 = p1;
        Console.WriteLine("Reference type Equals: {0}", p1.Equals(p3));

        int i = 10;
        int j = 10;
        Console.WriteLine("Value type Equals: {0}",i.Equals(j));
    }
}
```

```
Reference type Equals: True
Reference type Equals: True
Value type Equals: True
```

Override Equals – for inbuilt reference types

- Any type can override Object.Equals method to change the logic of how it checks for equality e.g. String, Delegate and Tuple do this to provide the value equality, even though these are the reference types.

```
public class GFG{
    static public void Main (){

        string s1 = "Avishek";
        string s2 = "Avishek";

        Console.WriteLine("Reference type Equals: {0}", s1.Equals(s2));
    }
}
```

```
Reference type Equals: True
```

Reference Equals

- Equals is an instance method that takes one parameter (which can be null). Since it is an instance method (must be invoked on an actual object), it can't be invoked on a null-reference. It can be overridden. Equals is also a static method on object which takes two parameters. One or both can then be null. Object.Equals(null,null)
- ReferenceEquals is a static method that takes two parameters, either / both of which can be null. Since it is static (not associated with an object instance), it will not throw a NullReferenceException under any circumstances. It cannot be overridden.

```
object o1 = null;
object o2 = new object();

//Technically, these should read object.ReferenceEquals for clarity, but this is redundant.
ReferenceEquals(o1, o1); //true
ReferenceEquals(o1, o2); //false
ReferenceEquals(o2, o1); //false
ReferenceEquals(o2, o2); //true

o1.Equals(o1); //NullReferenceException
o1.Equals(o2); //NullReferenceException
o2.Equals(o1); //false
o2.Equals(o2); //true
```

Difference between Equals and ==

- ==is an operator, that, in this case (object), behaves identically to ReferenceEquals. It will not throw a NullReferenceExceptioneither.

```
public class GFG{
    static public void Main (){
        Person p1 = new Person("Avishek", 10);
        Person p2 = new Person("Avishek", 10);

        //p1 and p2 points to different memory locations
        Console.WriteLine(p1.Equals(p2));   //False
        Console.WriteLine(p1==p2);          //False

        //Now p3 points to same memory location as p1.
        Person p3 = p1;
        Console.WriteLine(p1.Equals(p3));   //True
        Console.WriteLine(p1==p3);          //True

        int i = 10;
        int j = 10;
        Console.WriteLine(i.Equals(j));     //True
        Console.WriteLine(i==j);            //True

        object s1 = new string("Avishek");
        object s2 = new string("Avishek");
        Console.WriteLine(s1.Equals(s2));   //True : Equals is overriden by default for inbuilt reference types
        Console.WriteLine(s1==s2);          //False
    }
}
```

GetType()

- Returns the runtime object.

```
using System;

public class MyBaseClass {
}

public class MyDerivedClass: MyBaseClass {
}

public class Test
{
   public static void Main()
   {
      MyBaseClass myBase = new MyBaseClass();
      MyDerivedClass myDerived = new MyDerivedClass();
      object o = myDerived;
      MyBaseClass b = myDerived;

      Console.WriteLine("mybase: Type is {0}", myBase.GetType());
      Console.WriteLine("myDerived: Type is {0}", myDerived.GetType());
      Console.WriteLine("object o = myDerived: Type is {0}", o.GetType());
      Console.WriteLine("MyBaseClass b = myDerived: Type is {0}", b.GetType());
   }
}
// The example displays the following output:
//    mybase: Type is MyBaseClass
//    myDerived: Type is MyDerivedClass
//    object o = myDerived: Type is MyDerivedClass
//    MyBaseClass b = myDerived: Type is MyDerivedClass
```

ToString()

- Returns a string that represents the current object.

```
public class Person{
    public string name {get; set;}
    public int    age  {get; set;}
    public Person(string name, int age){
       this.name = name;
       this.age  = age;
    }
}

public class GFG{
    static public void Main (){
        Person p1 = new Person("Avishek", 10);
        Console.WriteLine(p1.ToString());
    }
}
```

```
Person:
```

```
public class Person{
    public string name {get; set;}
    public int    age  {get; set;}
    public Person(string name, int age){
       this.name = name;
       this.age  = age;
    }
    public override string ToString(){
        return base.ToString() + ": " + name.ToString() + " " + age.ToString();
    }
}

public class GFG{
    static public void Main (){
        Person p1 = new Person("Avishek", 10);
        Console.WriteLine(p1.ToString());
    }
}
```

```
Person: Avishek 10
```

GetHashCode()

- A hash code for the current object.

```
public class GFG{
    static public void Main (){
        Object obj = new Object();
        Type t = obj.GetType();

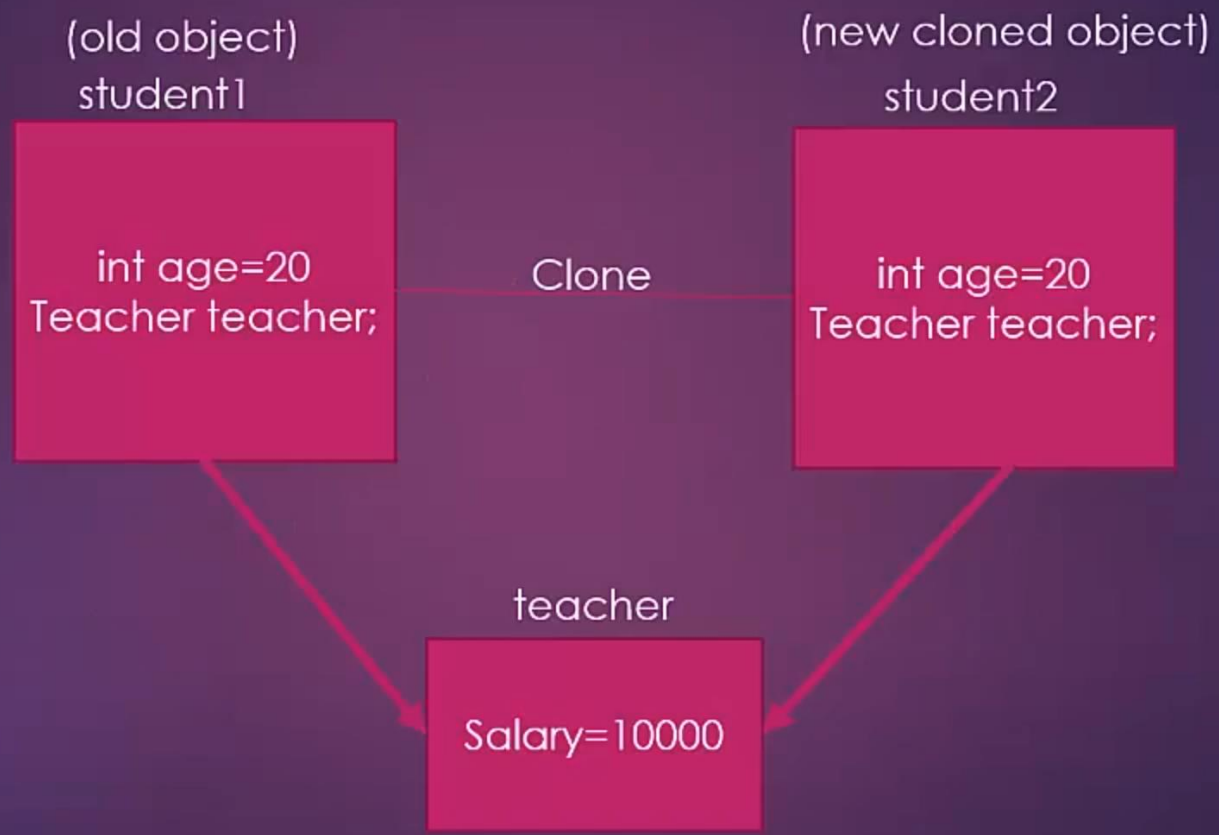
        // Display type and hash code
        Console.WriteLine("Type is :{0}", t);
        Console.WriteLine("Hash Code is :{0}", obj.GetHashCode());
    }
}
```

```
Type is :System.Object
Hash Code is :-1270152702
```

MemberWiseClone()

- Memberwise clone helps us to achieve shallow copy.
- Shallow copying is creating a new object and then copying the non-static fields of the current object.
- If the field is a value type then a bit by bit copy will be done
- If the field is reference type, then only the reference(memory) will be copied but the actual object referred to will not be copied.

Shallow Copy



student1 object and student2 are pointing to same teacher object

Ankpro

```
public class PersonalInfo{
    public string FirstName;
    public string LastName;
    public PersonalInfo(string fname, string lname){
        FirstName = fname;
        LastName = lname;
    }
}

public class Employee{
    public int EmpId;           //value type
    public string Department;   //reference type but is immutable. Hence while copying, a new string is created
    public PersonalInfo pi;     //reference type

    public Employee(int id, string dept, string fname, string lname){
        EmpId = id;
        Department = dept;
        pi = new PersonalInfo(fname, lname);
    }

    public Employee ShallowCopy(){
        return (Employee)this.MemberwiseClone();
    }
}

public class GFG{
    static public void Main (string []args){
        Employee emp1 = new Employee(100, "Finance", "Johnny", "Sinns");
        Employee emp2 = emp1.ShallowCopy();

        emp2.EmpId = 400;
        Console.WriteLine("emp1 id: {0}", emp1.EmpId);   //100
        Console.WriteLine("emp2 id: {0}", emp2.EmpId);   //400

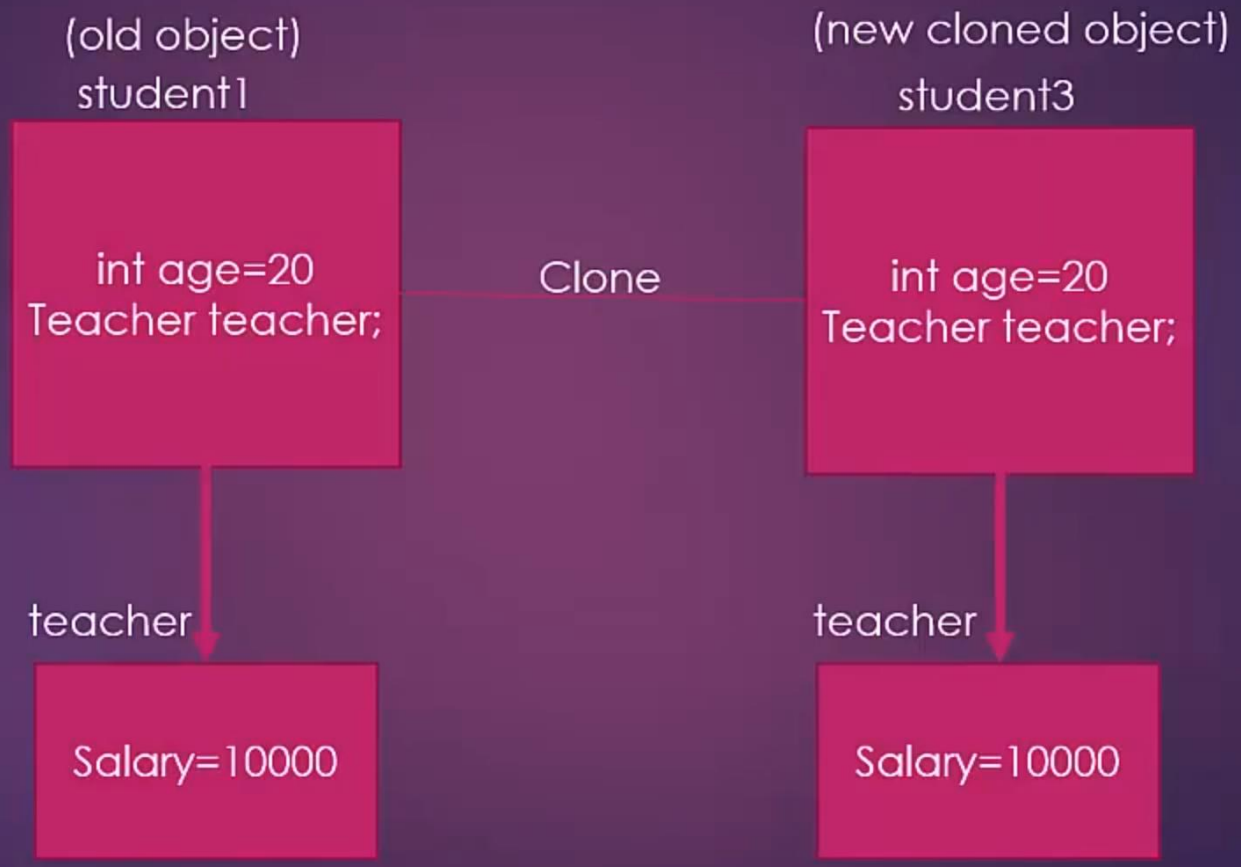
        emp2.pi.FirstName = "Danial";
        Console.WriteLine("emp1 Name: {0}", emp1.pi.FirstName);     //Danial ?? Because of shallow copy
        Console.WriteLine("emp2 Name: {0}", emp2.pi.FirstName);     //Danial

        emp2.Department = "Talent";
        Console.WriteLine("emp1 Name: {0}", emp1.Department);       //Finance
        Console.WriteLine("emp2 Name: {0}", emp2.Department);       //Talent
    }
}
```

Deep Copy

- Deep copy is creating a new object and then copying the non-static fields of the current object to the new object.
- If a field is of value type, then a bit by bit copy of the field is performed.
- If a field is of reference type, then the object that is referred to is copied, and the new reference is returned.

Deep Copy



student1 object and student3 are pointing to different teacher object

Ankpro

```
public class PersonalInfo{
    public string FirstName;
    public string LastName;
    public PersonalInfo(string fname, string lname){
        FirstName = fname;
        LastName = lname;
    }
}

public class Employee{
    public int EmpId;           //value type
    public string Department;   //reference type but is immutable. Hence while copying, a new string is created
    public PersonalInfo pi;     //reference type

    public Employee(int id, string dept, string fname, string lname){
        EmpId = id;
        Department = dept;
        pi = new PersonalInfo(fname, lname);
    }

    public Employee DeepCopy(){
        Employee emp= (Employee)this.MemberwiseClone();
        PersonalInfo empPi = new PersonalInfo(this.pi.FirstName, this.pi.LastName);
        emp.pi = empPi;
        return emp;
    }
}

public class GFG{
    static public void Main (string []args){
        Employee emp1 = new Employee(100, "Finance", "Johnny", "Sinns");
        Employee emp2 = emp1.DeepCopy();

        emp2.EmpId = 400;
        Console.WriteLine("emp1 id: {0}", emp1.EmpId);   //100
        Console.WriteLine("emp2 id: {0}", emp2.EmpId);   //400

        emp2.pi.FirstName = "Danial";
        Console.WriteLine("emp1 Name: {0}", emp1.pi.FirstName);     //Johnny
        Console.WriteLine("emp2 Name: {0}", emp2.pi.FirstName);     //Danial

        emp2.Department = "Talent";
        Console.WriteLine("emp1 Name: {0}", emp1.Department);       //Finance
        Console.WriteLine("emp2 Name: {0}", emp2.Department);       //Talent
    }
}
```

- Assignment operator does a shallow copy